

Distributed Systems Concerns in Practice

Fabrics and topology:

- Intra-node: PCIe, NVLink
- Inter-node: 25/100/200/400 Gbps Ethernet/InfiniBand; latencies 2–10× higher

Collectives vs parameter server:

- Collectives (NCCL/Gloo/MPI): bandwidth-efficient, symmetric
- PS: flexible, supports async, but can bottleneck at servers

Orchestration & launch:

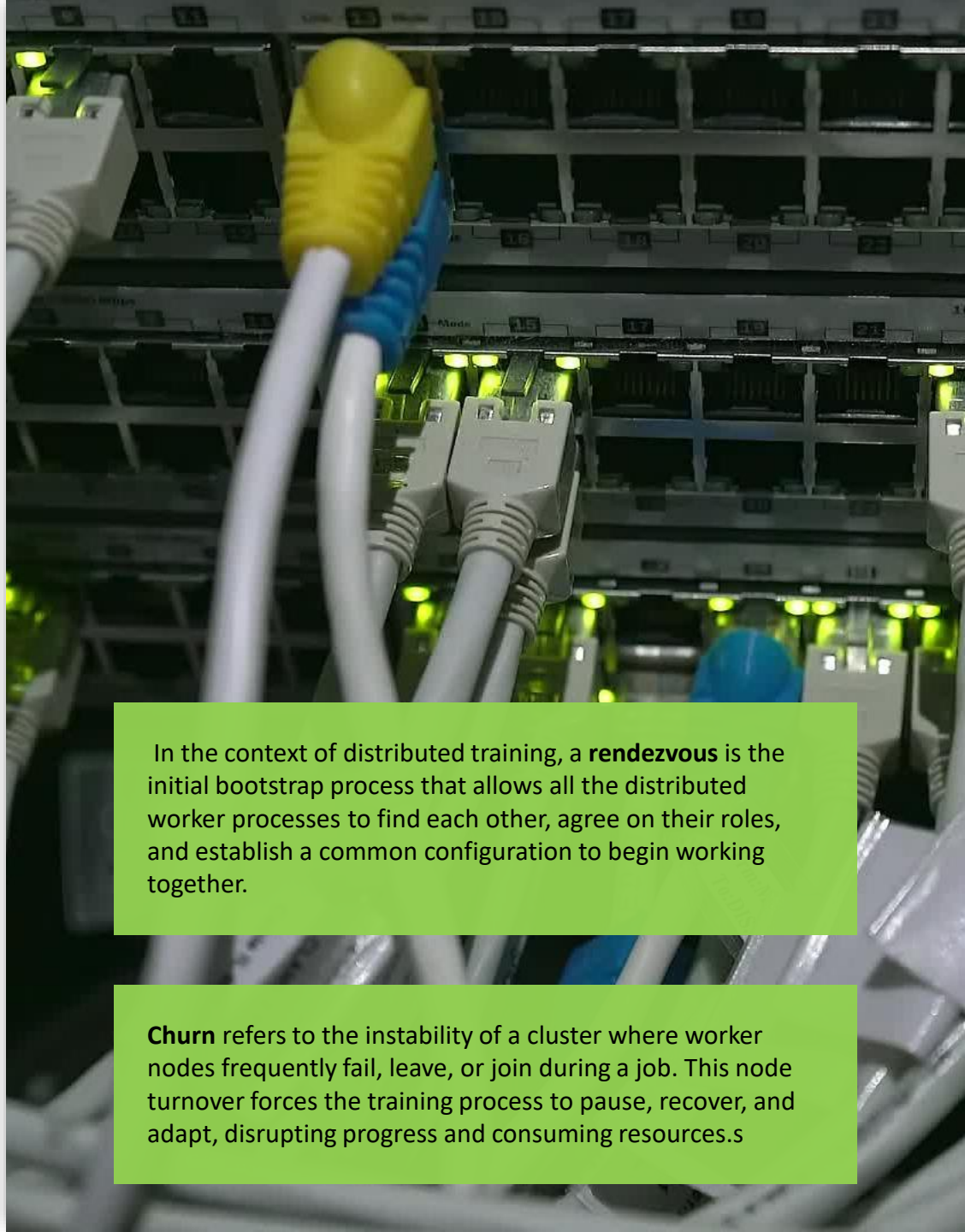
- SLURM/Kubernetes; ranks, world size, rendezvous; topology-aware placement

Reproducibility:

- Deterministic sharding and seeding; avoid nondeterministic kernels

Elasticity:

- Handle node churn; reshard data and adjust LR safely



In the context of distributed training, a **rendezvous** is the initial bootstrap process that allows all the distributed worker processes to find each other, agree on their roles, and establish a common configuration to begin working together.

Churn refers to the instability of a cluster where worker nodes frequently fail, leave, or join during a job. This node turnover forces the training process to pause, recover, and adapt, disrupting progress and consuming resources.

Sync vs Async vs Elastic Data Parallel

Synchronous all-reduce:

- Pros: stable convergence; single-node equivalence
- Cons: straggler-sensitive; per-step barrier

Parameter-server (async/stale-sync):

- Pros: more throughput; tolerant to slow clients
- Cons: stale gradients; careful LR tuning; potential accuracy hit

Elastic DP (synchronous with changing world size):

- World size adjusts at runtime; reshard data/checkpoints; adjust LR with effective batch

Guidance: prefer sync for accuracy-critical; use async/elastic when churn is inevitable



Reproducibility & Determinism at Scale

Seeds and sharding

- Set global and per-rank RNG seeds (Python, NumPy, framework, CUDA)
- Use a deterministic DistributedSampler so each rank gets a stable, unique shard

Deterministic ops

- Enable deterministic backends/flags
- Avoid or replace known nondeterministic kernels; document any exceptions

Floating-point non-associativity

- Different reduction orders/bucketization yield small numeric diffs
- Validate with tolerances (e.g., atol/rtol), not strict bitwise equality

Version and environment pinning

- Record container image, CUDA/cuDNN/NCCL, driver, framework versions
- Log code commit, configs/hyperparams, hardware topology and world size

Checkpoints

- Save model weights, optimizer state, RNG states (CPU/CUDA), and dataloader epoch/offset
- Include sharding metadata (DP/TP/PP groups, FSDP/ZeRO maps) for exact restarts
- Support elastic/changed world sizes with remapping info